

Images in Python

An image is usually represented as a three-dimensional array of 8-bit integers. NumPy arrays are the most commonly used library for this sort of data structure.

If you have an RGB image that is 480 pixels tall and 640 pixels wide, you will need a $480 \times 640 \times 3$ NumPy array.

There is a separate library (`imageio`) that:

- Reads an image file (like JPEG files) and creates a NumPy array.
- Writes a NumPy array to a file in standard image formats

Let's create a simply python program that creates a file containing an all-black image that is 640 pixels wide and 480 pixels tall. Create a file called `create_image.py`:

```
1 import NumPy as np
2 import imageio
3 import sys
4
5 # Check command-line arguments
6 if len(sys.argv) < 2:
7     print(f"Usage {sys.argv[0]} <outfile>")
8     sys.exit(1)
9
10 # Constants
11 IMAGE_WIDTH = 640
12 IMAGE_HEIGHT = 480
13
14 # Create an array of zeros
15 image = np.zeros((IMAGE_HEIGHT, IMAGE_WIDTH, 3), dtype=np.uint8)
16
17 # Write the array to the file
18 imageio.imwrite(sys.argv[1], image)
```

To run this, you will need to supply the name of the file you are trying to create. The extension (like `.png` or `.jpeg`) will tell `imageio` what format you want written. Run it now:

```
1 python3 create_image.py blackness.png
```

Open the image to confirm that it is 640 pixels wide, 480 pixels tall, and completely black.

1.1 Adding color

Now, let's walk through the image, pixel-by-pixel, adding some red. We will gradually increase the red from 0 on the left to 255 on the right.

```
1  import NumPy as np
2  import imageio
3  import sys
4
5  # Check command-line arguments
6  if len(sys.argv) < 2:
7      print(f"Usage {sys.argv[0]} <outfile>")
8      sys.exit(1)
9
10 # Constants
11 IMAGE_WIDTH = 640
12 IMAGE_HEIGHT = 480
13
14 # Create an array of zeros
15 image = np.zeros((IMAGE_HEIGHT, IMAGE_WIDTH, 3), dtype=np.uint8)
16
17 for col in range(IMAGE_WIDTH):
18
19     # Red goes from 0 to 255 (left to right)
20     r = int(col * 255.0 / IMAGE_WIDTH)
21
22     # Update all the pixels in that column
23     for row in range(IMAGE_HEIGHT):
24         # Set the red pixel
25         image[row, col, 0] = r
26
27 # Write the array to the file
28 imageio.imwrite(sys.argv[1], image)
```

When you run the function to create a new image, it will be a fade from black to red as you move from left to right:



Now, inside the inner loop, update the blue channel so that it goes from zero at the top to 255 at the bottom:

```
1      # Update all the pixels in that column
2      for row in range(IMAGE_HEIGHT):
3
4          # Update the red channel
5          image[row,col,0] = r
6
7          # Blue goes from 0 to 255 (top to bottom)
8          b = int(row * 255.0 / IMAGE_HEIGHT)
9          image[row,col,2] = b
10
11 imageio.imwrite(sys.argv[1], image)
```

When you run the program again, you will see the color fades from black to blue as you go down the left side. As you go down the right side, the color fades from red to magenta.



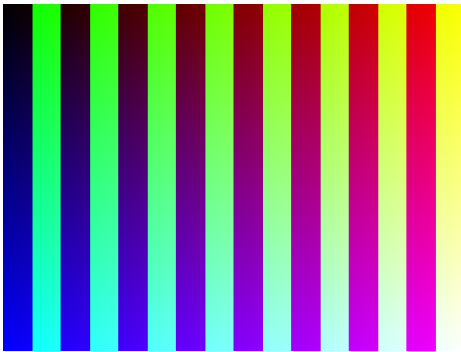
Notice that red and blue with no green looks magenta to your eye.

Next, let's add some stripes of green:

```
1 import NumPy as np
2 import imageio
```

```
3 import sys
4
5 # Check command line arguments
6 if len(sys.argv) < 2:
7     print(f"Usage {sys.argv[0]} <outfile>")
8     sys.exit(1)
9
10 # Constants
11 IMAGE_WIDTH = 640
12 IMAGE_HEIGHT = 480
13 STRIPE_WIDTH = 40
14 pattern_width = STRIPE_WIDTH * 2
15
16 # Create an image of all zeros
17 image = np.zeros((IMAGE_HEIGHT, IMAGE_WIDTH, 3), dtype=np.uint8)
18
19 # Step from left to right
20 for col in range(IMAGE_WIDTH):
21
22     # Red goes from 0 to 255 (left to right)
23     r = int(col * 255.0 / IMAGE_WIDTH)
24
25     # Should I add green to this column?
26     should_green = col % pattern_width > STRIPE_WIDTH
27
28     # Update all the pixels in that column
29     for row in range(IMAGE_HEIGHT):
30
31         # Update the red channel
32         image[row,col,0] = r
33
34         # Should I add green to this pixel?
35         if should_green:
36             image[row,col,1] = 255
37
38         # Blue goes from 0 to 255 (top to bottom)
39         b = int(row * 255.0 / IMAGE_HEIGHT)
40         image[row,col,2] = b
41
42 imageio.imwrite(sys.argv[1], image)
```

When you run this version, you will see the previous image in half the stripes. In the other half, you will see that green fades to cyan down the left side, and yellow fades to white down the right side.



1.2 Using an existing image

imageio can also be used to read in any common image file format. Let's read in an image and save each of the red, green, and blue channels out as its own image.

Create a new file called `separate_image.py`:

```
1  import imageio
2  import sys
3  import os
4
5  # Check command line arguments
6  if len(sys.argv) < 2:
7      print(f"Usage {sys.argv[0]} <infile>")
8      sys.exit(1)
9
10 # Read the image
11 path = sys.argv[1]
12 image = imageio.imread(path)
13
14 # What is the filename?
15 filename = os.path.basename(path)
16
17 # What is the shape of the array?
18 original_shape = image.shape
19
20 # Log it
21 print(f"Shape of {filename}:{original_shape}")
22
23 # Names of the colors for the filenames
24 colors = ['red', 'green', 'blue']
25
26 # Step through each of the colors
27 for i in range(3):
28
29     # Create a new image
30     newimage = np.zeros(original_shape, dtype=np.uint8)
```

```
31
32     # Copy one channel
33     newimage[:, :, i] = image[:, :, i]
34
35     # Save to a file
36     new_filename = f"{colors[i]}_{filename}"
37     print(f"Writing {new_filename}")
38     imageio.imwrite(new_filename, newimage)
```

Now, you can run the program with any common RGB image type:

```
1 python3 separate_image.py dog.jpg
```

This will create three images: red_dog.jpg, green_dog.jpg, and blue_dog.jpg.

This is a draft chapter from the Kontinua Project. Please see our website (<https://kontinua.org/>) for more details.

Answers to Exercises

